

CP Drid's Guide to Kubernetes RBAC and Pod Security: Least Privilege Inside the Cluster

By CP Drid | Sec Fundamentals

Abstract: A modest foothold plus over-broad permissions is how a single compromised pod becomes control of an entire Kubernetes cluster. This paper examines how to enforce least privilege inside the cluster. It covers role-based access control and the misconfigurations that turn a foothold into takeover, service account and token hygiene, the Pod Security Standards enforced through Pod Security Admission, policy-as-code enforcement with admission controllers, and auditing cluster authorization into a SIEM. It closes with a hardening checklist.

Introduction

The earlier papers in this collection looked outward and sideways, at the cluster edge and at pod-to-pod traffic. This one looks inward, at authorization: who and what is allowed to do what inside the cluster. It is the layer attackers love, because a modest foothold combined with over-broad permissions is how a single compromised pod becomes control of the whole cluster. Least privilege is the antidote, and Kubernetes gives you three tools to enforce it: role-based access control, disciplined service accounts, and the Pod Security Standards. Used together, they keep a breach small and contained instead of catastrophic.

RBAC Is the Cluster's Authorization Surface

Role-based access control is how Kubernetes decides whether a given identity may perform a given action on a given resource. It rests on four pieces. Roles and ClusterRoles define permissions, a set of verbs such as get, list, create, and delete against resources such as pods, secrets, and deployments. A Role is scoped to a single namespace, while a ClusterRole applies cluster-wide. RoleBindings and ClusterRoleBindings then attach those permissions to subjects, which are users, groups, or service accounts. The security discipline is simple to state and easy to neglect: grant the minimum verbs on the minimum resources in the minimum scope. Prefer a namespaced Role and RoleBinding over a cluster-wide grant whenever the workload only operates in one namespace, because scope is itself a control.

The RBAC Mistakes That Become Breaches

Most cluster takeovers trace back to a handful of RBAC habits. Wildcards are the first: a Role that grants all verbs on all resources hands the holder the keys to that scope, and the same wildcard in a

ClusterRole hands over the cluster. Binding anything other than a cluster operator to the built-in cluster-admin role is the second, and it happens far more often than it should as a shortcut to make something work. Beyond the obvious, a few specific verbs are escalation primitives and deserve special scrutiny: escalate and bind let a subject grant itself more than it already has, impersonate lets it act as another identity, and broad access to secrets hands over the credentials those secrets protect. The ability to create pods or to use pods/exec is quietly powerful too, because a subject who can schedule or enter a pod can often mount sensitive tokens or reach the node. Audit for these deliberately rather than assuming they are safe.

Service Accounts and Token Hygiene

Every pod runs as a service account, and by default it is the namespace default account, with its token automatically mounted into the container. That default is the problem. If a pod is compromised, whatever its service account can do through RBAC, the attacker can now do. Three practices tighten this. First, give each workload its own dedicated service account with a least-privilege binding, rather than sharing the default. Second, turn off token automounting where a workload does not call the Kubernetes API at all, by setting automountServiceAccountToken to false, so there is no token to steal. Third, rely on short-lived, audience-bound projected tokens, which modern Kubernetes issues through the TokenRequest API, instead of the old long-lived token secrets that never expired. A token that lives for an hour and only works against one audience is a far smaller prize than a permanent, general-purpose credential sitting in a mounted secret.

Pod Security Standards and Pod Security Admission

Controlling what a pod is allowed to be is as important as controlling what it can call. The Pod Security Standards define three profiles. Privileged is unrestricted and meant only for trusted, infrastructure-level workloads. Baseline blocks the known privilege-escalation vectors, such as privileged containers, host namespaces, and host path mounts. Restricted is the hardened profile: it requires containers to run as non-root, forbids privilege escalation, drops all Linux capabilities, and enforces a default seccomp profile. These profiles are enforced by Pod Security Admission, the built-in controller that replaced the removed PodSecurityPolicy feature. You apply it per namespace with labels, and it runs in three modes: enforce to reject violating pods, audit to record them, and warn to notify the user. A sound rollout applies Restricted in warn and audit first to find what breaks, then flips to enforce once the workloads comply.

Enforce Policy as Code

Pod Security Admission is deliberately narrow. When you need rules it does not cover, reach for a policy-as-code admission controller such as Kyverno or OPA Gatekeeper, or the built-in CEL-based validating admission policies in newer Kubernetes releases. These let you enforce organization-specific guardrails at admission time: allow images only from approved registries, forbid the latest tag, require CPU and memory limits on every container, mandate specific security context settings, or require particular labels for ownership and cost tracking. Like Pod Security Admission, these controllers can run in audit before enforce, so you measure impact before you start rejecting deployments. Policy as code also means your guardrails live in version control, are reviewed like any other change, and cannot drift silently.

Human Access: Real Identities, Not Shared Certs

RBAC decides what an identity may do, but it only matters after that identity is established, so human authentication is the front half of the same control. Kubernetes does not manage user accounts itself, which means the common shortcut is to hand out long-lived client certificates or copy a shared admin kubeconfig around a team. Both are dangerous: a client certificate cannot be easily revoked, and a shared credential erases any trace of who actually did what. The stronger pattern is to integrate the API server with your organization's identity provider through OIDC, so people log in as themselves with single sign-on and multi-factor authentication, and their group membership drives their RBAC. Then scope those human grants the same way you scope workloads, per namespace and per team, and reserve cluster-admin for a named few. Real identities also make the audit trail meaningful, because every action ties back to a person rather than to a shared key.

Audit and Detect

Least privilege is not a one-time configuration, it is a posture you have to keep verifying. Start with the tooling built into kubectl: `auth can-i` answers whether a given identity may perform a given action, and community tools such as `who-can` and `rbac-lookup` invert the question to show who can do something dangerous, like reading secrets or creating pods across the cluster. Then close the loop with the Kubernetes audit log, shipped to your SIEM using the same pipeline described in the earlier papers. Alert on the events that matter: creation of a `ClusterRoleBinding`, any binding to cluster-admin, exec into a running pod, and reads against sensitive secrets. Pair this with a periodic review of every `ClusterRoleBinding` and every privileged workload, because permissions accumulate over time and nobody removes them unless someone looks.

Least Privilege Hardening Checklist

- Grant the minimum verbs on the minimum resources in the minimum scope, and prefer namespaced Roles over ClusterRoles.
- Eliminate wildcard verbs and resources in Roles and ClusterRoles.
- Bind cluster-admin only to genuine cluster operators.
- Audit for the escalation verbs: escalate, bind, and impersonate.
- Restrict who can read secrets, create pods, and use pods/exec.
- Give every workload a dedicated least-privilege service account, not the default.
- Disable token automounting for workloads that do not call the API.
- Use short-lived, audience-bound projected tokens instead of legacy token secrets.
- Apply the Restricted Pod Security Standard per namespace, rolled out through warn and audit before enforce.
- Add policy-as-code guardrails for image sources, resource limits, and security context.
- Ship the Kubernetes audit log to the SIEM and alert on RBAC changes, cluster-admin bindings, exec, and secret access.
- Review every ClusterRoleBinding and privileged workload on a schedule.

Grant the least, verify often, and keep a foothold from becoming a takeover.

Stay least-privileged and stay secure. CPerk